

2021 BIT Part 01

① 1. {

2. {[

3. {[]}

4. {[

5. {[[

6. {[[[[

7. {[[[[] } ← Ans → d

8. {[[] }

9. {[() }

10. {[

11. {[[

12. {[[] }

13. {[() }

14. { } }

$$\{ [[[[[]]]] ([]) \}$$

1 2 3 4 5 6 7 8 9 10 11 12 13 14

② A queue follow the FIFO.

→ Good for performing tasks in order.

a) CPU Scheduling ✓

b) Backtracking ✗ (stack)

c) Async data transfer ✓

d) Implementing a dict ✗ (Trees...)

e) Reversing characters ✗ (stack)

Ans → b, d, e

③ SQPRT

TRP

Ans → a

④ 34, 8, 64, 31, 32, 21, 9 $\leftarrow$ Initial.

Pass 1 34, 8, 64, 31, 32, 21, 9

Pass 2 8, 34, 64, 31, 32, 21, 9

Pass 3 8, 34, 64, 31, 32, 21, 9

8, 31, 34, 64, 32, 21, 9

Ans $\rightarrow$ b

⑤ Worst case - Array is in reverse order.

worst case complexity $O(n^2)$

Big-O notation is independent of specific constants like 10, or 100.

Ans $\rightarrow$ a, c

⑥ In this question, the first element of the array / sub-array is the pivot.

Initial $\rightarrow$ 7, 67, 20, 16, 9, 4, 11, 75, 5, 89

1st Pivot $\rightarrow$ 7

Pass 1 4, 5, 7, 67, 20, 16, 9, 11, 75, 89

Next pivots

Partition 2 $\rightarrow$ 4, 5, 7, 20, 16, 9, 11, 67, 75, 89
 next one remaining. next next

Pg

Partition 3 $\rightarrow$ 4, 5, 7, 16, 9, 11, 20, 67, 75, 89
 next next

Partition 4 $\rightarrow$ 4, 5, 7, 9, 11, 16, 20, 67, 75, 89
 next

Partition 5 $\rightarrow$ 4, 5, 7, 9, 11, 16, 20, 67, 75, 89

Final $\rightarrow$

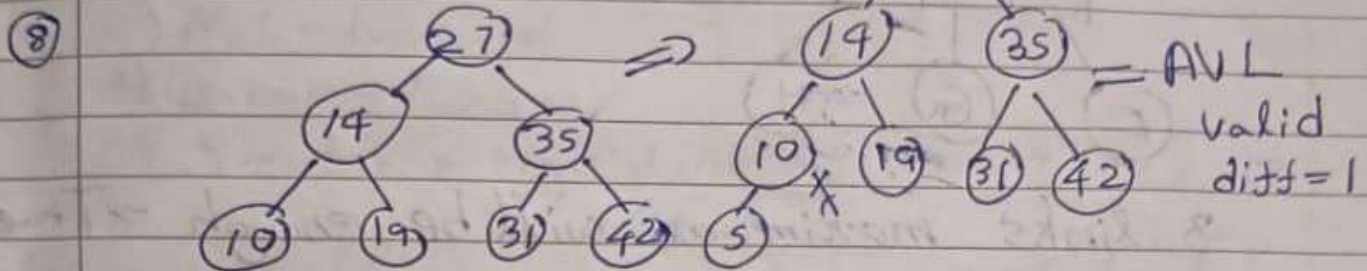
(6) Initial $\rightarrow$ 7, 67, 20, 16, 9, 4, 11, 75, 5, 89
 low high
 $i = -1$ Pivot, j
 4 < 7 $\rightarrow$ i++ and swap

$\Downarrow$
 4, 67, 20, 16, 9, 7, 11, 75, 5, 89
 i Pivot, j 5 < 7
 i++ and swap

$\Downarrow$
 4, 5, 20, 16, 9, 7, 11, 75, 67, 89
 i pivot 0 j reached high
 i++ and swap
 Pivot

$\Downarrow$
 4, 5, 7, 16, 9, 20, 11, 75, 67, 89
 Pivot 1 i j
 Pivot 2
 7

7) Ans $\rightarrow$ (d) 6

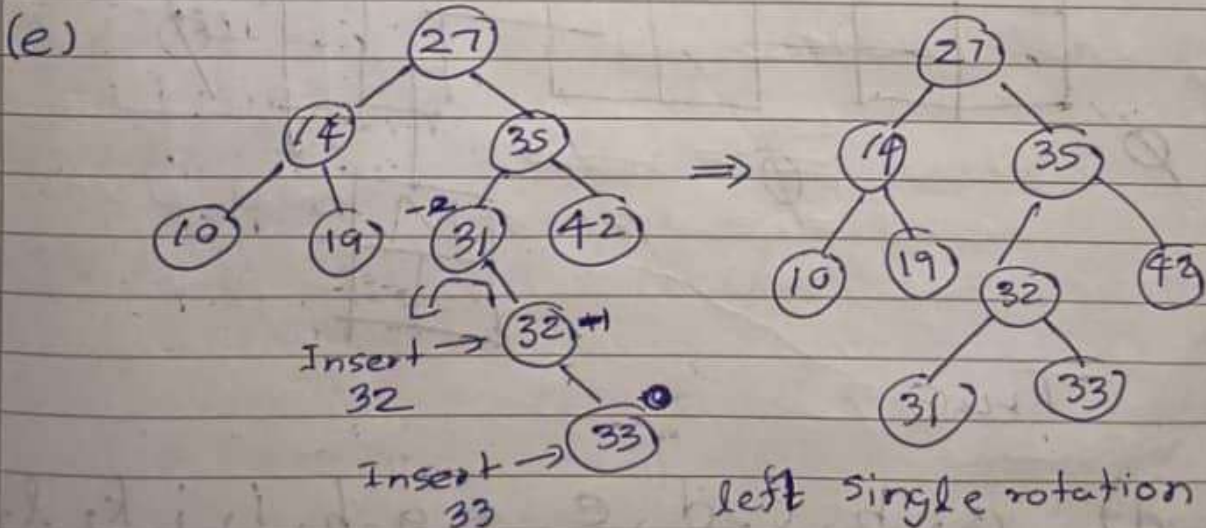


a) Binary Tree $\checkmark$
Binary search tree $\checkmark \rightarrow$ False

b) Height difference of +4, 0, -1 is valid.
 $\rightarrow$ False.

c) Siblings of 14 are 35. $\rightarrow$ False.

d) Height of tree is 2 $\rightarrow$ True.



$\rightarrow$ True.

Ans $\rightarrow$ d, e

✓ ✓ ✓
 (4), 5, (7), 9, 16, 20, 11, 75, 67, 89
 i Pivot j
 ↓ < 16
 i++ and swap

4, 5, 7, 9, 11, 20, 16, 75, 67, 89

i → Pivot ↑ j → reached high i++ and swap pivot

$(4), 5, (7), 9, 11, (16), 20, 75, 67, 89$
 i j
 Pivot 3 Pivot 4

4, 5, 7, 9, 11, 16, 20, 75, 67, 89
 i j
 Pivot 4 reached high
 i++ and
 swap pivot

$(4), 5, (7), (9), 11, (16), (20), 75, 67, 89$
 $i \quad \uparrow \quad j$
 Pivot s

$67 < 75$
 $i++$ and swap

4, 5, 7, 9, 11, 16, 20, 67, 75, 89

i Pivot j

reache high

it + and swap

pivot

Ans $\rightarrow$ b, c

Pivots $\rightarrow 7, 4, 16, 9, 20, 75$

- (10) a) ~~Stack~~ - Linear
 b) Red-Black tree - X
 c) AVL - tree - X
 d) Queue - Linear
 e) Circular array - Linear Ans $\rightarrow$ a, d, e

(11) $\rightarrow$ Normally a heap is a complete ^{binary} tree.
 So, automatically it is a Balanced tree too.

$\rightarrow$ Heap is a binary tree and it doesn't follow BST rules.

$\rightarrow$ Min Heap - For any given node P, the value of P is less than or equal to the value of its children.

$\rightarrow$ Max Heap - Vice-versa.

Question says parent is less than its child.

$\rightarrow$ ~~Ans~~ Ans $\rightarrow$ Min Heap.
 and DFS

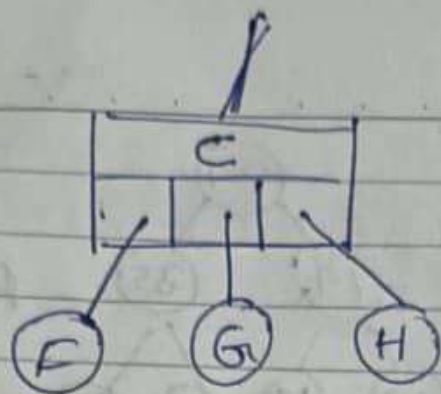
(12) BFS in Graphs $\rightarrow$ start from unvisited node.

$\rightarrow$ Visit all nodes reachable from it.

$\rightarrow$ All visited node belong to a connected component.

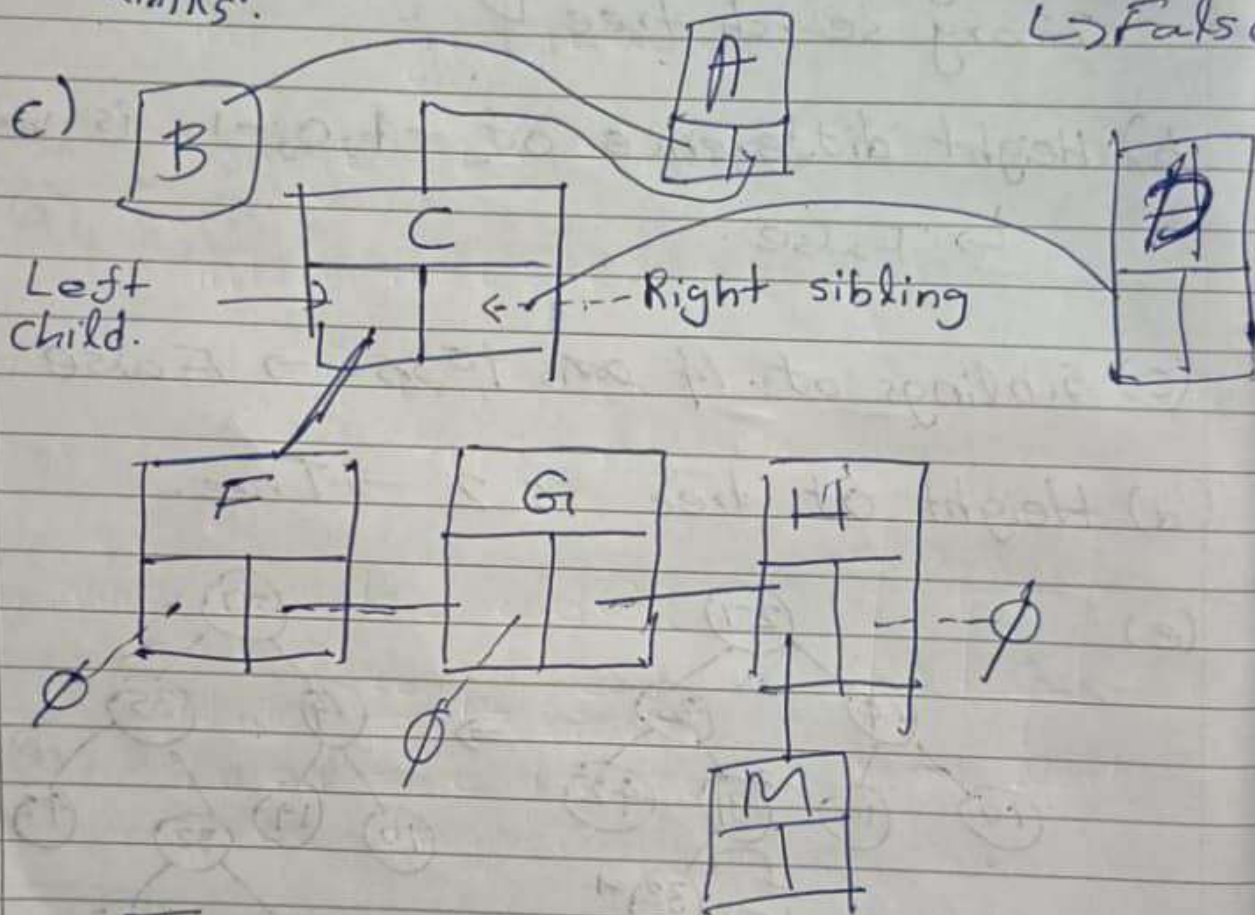
Ans $\rightarrow$ a, b.

9) a)



3 links maximum will be enough $\rightarrow$ True

b) To implement this we need at least 3 links. $\rightarrow$ False



d) a, b, c, d, e, f, g, h, i, j, k, l, m
 l is missing, so invalid.

$\rightarrow$ False.

d) C's ancestor is A $\rightarrow$ A $\rightarrow$ A $\rightarrow$ A, a, m

(17) Fibonacci(n) {

IF $n == 0$ THEN
RETURN 0

IF $n == 1$ THEN
RETURN 1

} Base case.

ELSE

RETURN Fibonacci($n-1$) + Fibonacci($n-2$)

ENDIF

}

Fibonacci(4)

↳ Fibonacci(3) + Fibonacci(2)

↳ Fibonacci(2) + Fibonacci(1)

↳ Fibonacci(1) +
Fibonacci(0)

↳ Fibonacci(1) + Fibonacci(0)

↓
1

↓
0

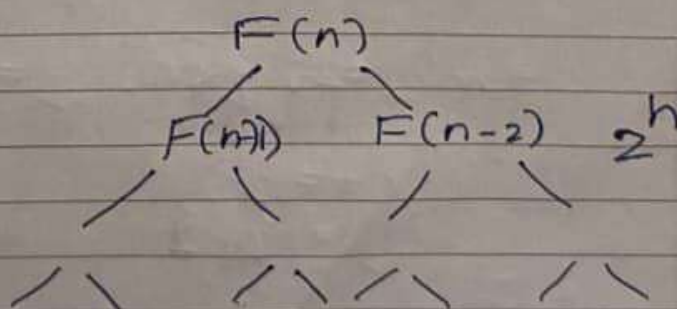
$$F(0) = 0$$

$$F(1) = 1$$

$$F(2) = 1$$

$$F(3) = 2$$

$$F(4) = 3$$



Cost function $\Rightarrow T(n) = T(n-1) + T(n-2)$

$$\begin{aligned} \text{Total} &= 2^0 + 2^1 + 2^2 + \dots + 2^n + 1 \\ &= 2^{n+1} - 1 \end{aligned}$$

$$O(2^n) //$$

a) Recursive func are not fast compared to iterations.

b) Recursive func takes ~~more~~ space.

c) Recursive methods can ^{always} be replaced by a non-recursive method.

d) ✓

e) stack is used to handle recursions.

Ans $\rightarrow$ b, c, d

(13) Ans $\rightarrow$ b)

(14) Java X

(15) Ans $\rightarrow$ c, d, e.

(16) We need a 2D array for this.

```
int array[2][2];
```

```
array[0][0] = 60;
```

```
int marks[2][2] = {
```

```
{ 45, 55 },
```

```
{ 34, 78 }
```

```
};
```

(16) void delete_head() {

```
Node* temp = head;
```

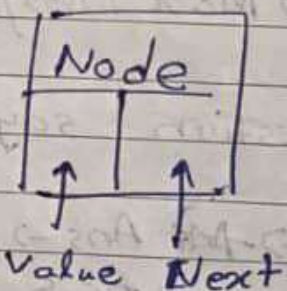
```
head = head  $\rightarrow$  next;
```

```
free(temp);
```

```
}
```

Ans $\rightarrow$ a

struct



⑮ ~~public static String reverse(str)~~

void reverse(char *str) {

if (*str == '\0') {
return;

}

reverse(str+1);

printf("%c", *str);

}

In Java;

reverse(string.substring(1))
+ string.charAt(0);

Reverse('MAN')

↳ Reverse('AN') + M

↳ Reverse('N') + A

↳ Reverse('\0') + N

↳ Return --



NAM

⑲ $O(\log n)$ Ans → d

⑳ Pre → Root → Left → Right.

In-order

Post → Left → Root → Right.

Post → Left → Right → Root

Outer loop

↳ for ($p = n/2$; $p < n$; $p++$)

$$h = n - \frac{n}{2} = \frac{n}{2} \leftarrow \text{impact } O(n)$$

Inner loop

↳ for ($q = 2$; $q < n$; $q = q * 2$) {

$r += q$;

$q = q * 2$;

}

$$q = 2 \times 2^k$$

$$q < n$$

$$2^k < n$$

$$\log_2 2^k < \log_2 n$$

$$k \log_2 2 < \log_2 n$$

$$k < \log_2 n$$

$$T(n) = \frac{n}{2} \times \log_2 n \Rightarrow O(n \log n)$$

Ans $\rightarrow C$

25) not in syllabus

Pre $\rightarrow P, Q, R, S, X, Y, W, Z$

In-Order $\rightarrow Q, P, R, X, S, W, Y, Z, R$

Post $\rightarrow Q, X, W, Z, Y, S, R, P$

Ans $\rightarrow a$

(21) After comparing weights;

Shortest path $\Rightarrow S \rightarrow C \rightarrow A \rightarrow B$

Ans $\rightarrow b$

(22) Bellman-Fort Algo is not there in your syllabus.

(23) $O(n^2)$

Ans $\rightarrow b$

(24) int $p, q, r = 0$

for ($p = n/2; p < n; p++$) {

for ($q = 2; q < n; q = q^2$) {

$r += q;$

$q = q * 2$

}

}